

Duale Hochschule Baden-Württemberg Stuttgart

Travel Assistant

Eine cloud-native KI-Reiseplanungsanwendung mit
automatisiertem Deployment, Monitoring und Disaster Recovery

Schriftliche Ausarbeitung zur Projektarbeit

Verfasser:	Timothy Kugler, 2348056
Modul:	Cloud Computing II
Abgabedatum:	3. Mai 2026
Repository:	github.com/timmykug/travel-assistant

Inhaltsverzeichnis

1	Einleitung	3
1.1	Kontext und Motivation	3
1.2	Zielsetzung	3
1.3	Problemstellung	3
2	Technische Grundlagen	4
2.1	Cloud-Service-Modelle	4
2.2	Eingesetzte Technologien	4
2.3	Begründung der Technologieauswahl	5
3	Architektur und Umsetzung	6
3.1	Gesamtarchitektur	6
3.2	Request-Pfad und Deployment-Flüsse	7
3.3	Deployment-Modell	8
3.4	Anwendungsumfang (Frontend und Backend)	8
3.5	Monitoring-Stack	9
4	Fokus-Feature: Disaster Recovery	10
4.1	Leitidee: zwei Recovery-Pfade	10
4.2	Compute-Recovery über die MIG	10
4.3	Datenrecovery	10
5	Ergebnisse und Diskussion	14
5.1	Rückblick auf den umgesetzten Systemumfang	14
5.2	Testumfang und Methodik	14
5.3	Grenzen der Lösung	15
5.4	Ausblick und Handlungsempfehlungen	15
6	Fazit	16
7	Anhang	16
7.1	Kostenrechnung	16
7.2	Testabdeckung	17
	Referenzen	18

1 Einleitung

1.1 Kontext und Motivation

Die Planung einer Reise ist auch im Jahr 2026 trotz einer Vielzahl spezialisierter Online-Dienste aufwändig. Informationen über Ziele, Aktivitäten und Zeitpläne liegen verteilt vor und müssen von Reisenden manuell zusammengeführt werden. Gleichzeitig haben sich generative Künstliche Intelligenz (KI)-Modelle zu einer tragfähigen Basis für konversationelle Assistenzsysteme entwickelt. Vor diesem Hintergrund steht die Projektidee eines dialogorientierten Reiseassistenten, der Nutzerinnen und Nutzern bei der Planung ganzer Reisen hilft, Gespräche speichert und personalisierte Vorschläge liefert.

Die Anwendung ist bewusst eher als Demonstrator für Cloud-Architektur und -Betrieb konzipiert denn als Feature-vollständige Produktivlösung. Der Fokus dieser Arbeit liegt stattdessen auf der *Cloud-nativen* Umsetzung: dem Entwurf einer robusten, skalierbaren und wartbaren Architektur, die im Rahmen des Fokus-Features *Disaster Recovery* sowohl Ausfälle als auch Datenverluste adressiert. Der Projektkontext umfasst die Nutzung aller drei Cloud-Ebenen (Infrastructure as a Service (IaaS), Platform as a Service (PaaS), Software as a Service (SaaS)), eine automatisierte Infrastrukturbereitstellung mit Terraform, eine Konfigurationsautomatisierung mit Ansible sowie ein erkennbares Monitoring-Konzept. Der vorliegende Projektbericht beschreibt die Umsetzung des entsprechenden Systems eines *Travel Assistant* auf der Google Cloud Platform (GCP).

1.2 Zielsetzung

Ziel des Projekts war die Entwicklung einer containerbasierten Web-Anwendung, die folgende Eigenschaften erfüllt:

- **Funktional:** ein KI-basierter Reiseassistent mit Nutzerkonten, persistenten Konversationen und speicherbaren Reiseplänen.
- **Architektonisch:** Einsatz aller drei Cloud-Ebenen (SaaS, PaaS, IaaS) mit klarer Verantwortungstrennung.
- **Betrieblich:** vollständig automatisierte Bereitstellung über Terraform, Ansible und GitHub Actions mit reproduzierbaren Konfigurationen.
- **Resilient:** ein Fokus-Feature *Disaster Recovery*, das sowohl Compute-Ausfälle über eine Managed Instance Group als auch Datenverluste in Firestore über ein Backup/Restore-Konzept behandelt.
- **Beobachtbar:** Monitoring mit Prometheus, Grafana und Loki, einschließlich eines dedizierten Integritätschecks für die Datenbank.

1.3 Problemstellung

Architekturprägend waren vier Kernfragen: wie ein stabiler öffentlicher Endpunkt trotz selbstheilender Virtual Machines (VMs) entsteht, wie App- und Datenfehler sichtbar gemacht werden, wie eine identische Wiederherstellung der Anwendung automatisiert abläuft und wie Datenstände gesichert und zurückgespielt werden. Die folgenden Kapitel entwickeln die Lösung von den technologischen Grundlagen (Kapitel 2) über die Architektur und Umsetzung (Kapitel 3) bis hin zum Fokus-Feature *Disaster Recovery* (Kapitel 4), einer Diskussion der Ergebnisse (Kapitel 5) und einem Fazit (Kapitel 6).

2 Technische Grundlagen

2.1 Cloud-Service-Modelle

Die folgende Tabelle fasst das im Projekt gewählte Mapping auf die drei Cloud-Service-Modelle zusammen.

Ebene	Im Projekt verwendet
SaaS	Google Gemini Application Programming Interface (API) (Gemini 3.1 Flash-Lite) für die KI-Antworten, GitHub Actions für die Continuous Integration/Continuous Deployment (CI/CD)-Automatisierung.
PaaS	Google Firestore (Not only SQL (NoSQL)), Google Artifact Registry für Docker-Images, Google Cloud Scheduler für geplante Backups, Google Cloud Storage für Terraform-State, App-Konfigurationen und Firestore-Backups, Google Secret Manager für Laufzeit-Secrets.
IaaS	Google Compute Engine (App-Managed Instance Group (MIG) + Monitoring-VM), Load Balancer, Persistent Disks.

Tabelle 1: Zuordnung der Projekt-Komponenten zu den Cloud-Service-Modellen.

2.2 Eingesetzte Technologien

- **Backend:** Python 3.13 mit FastAPI (`0.115`), Pydantic v2 und Uvicorn als ASGI-Server. E-Mail/Passwort-Authentifizierung mit serverseitig ausgestellten JWTs über `python-jose`, Prometheus-Metriken über den `prometheus-fastapi-instrumentator`.
- **Frontend:** React + Vite, ausgeliefert als statische Dateien über Nginx. Das Frontend hält das JSON Web Token (JWT) im Browser und sendet es als Bearer-Token an die API.
- **Datenhaltung:** Google Firestore (Native Mode) im Multi-Dokumenten-Modell (siehe Tabelle 2).

Collection	Inhalt
<code>users/{uid}</code>	Nutzerprofil (<code>display_name</code> , <code>email</code>)
<code>users/{uid}/conversations/{id}</code>	Chatverläufe mit dem Assistenten
<code>users/{uid}/trips/{id}</code>	Gespeicherte Reisepläne
<code>rate_limits/{uid}</code>	Tägliches Gemini-Request-Kontingent je Nutzer
<code>analytics/daily_usage</code> , <code>analytics/system</code>	Aggregierte Metriken und Seed-Markierung

Tabelle 2: Firestore-Datenmodell.

- **KI-Modell:** Google Gemini API mit *Gemini 3.1 Flash-Lite* für die Reiseassistenten; gewählt gegenüber 2.5 Flash wegen besserer Geschwindigkeits- und Intelligenzwerte [1] bei gleichem Free-Tier-Kontingent [2].
- **Container und Orchestrierung:** Docker + Docker Compose, ein eigener Nginx als Reverse Proxy pro App-VM.
- **Infrastructure as Code:** Terraform (Provider `hashicorp/google ~> 5.0`) für Netzwerk, VMs, Load Balancer, Firestore, Identity and Access Management (IAM) und Cloud Scheduler.
- **Konfigurationsmanagement:** Ansible (Playbook + Rolle) ausschließlich für die persistente Monitoring-VM.

- **CI/CD:** GitHub Actions (Build, Push, `terraform apply`, Ansible).
- **Monitoring:** Prometheus mit Google Compute Engine (GCE)-Service-Discovery, Grafana mit provisionierten Dashboards, Loki mit Promtail als Log-Shipper, Alertmanager für Benachrichtigungen, Blackbox Exporter für den Datenbank (DB)-Integritätscheck und Node Exporter für Host-Metriken.

2.3 Begründung der Technologieauswahl

Die Google Cloud Platform war durch die Projektvorgabe gesetzt. Die Auswahl der konkreten GCP-Dienste folgt jedoch der Architektur: Compute Engine bildet die IaaS-Schicht für App- und Monitoring-VMs, Firestore übernimmt als verwalteter PaaS-Dienst die persistente Datenerhaltung, Artifact Registry speichert die Container-Images, Cloud Storage hält Terraform-State, App-Konfigurationen und Backup-Artefakte, Cloud Scheduler stößt die geplanten Firestore-Exporte an, und Secret Manager hält die Laufzeit-Secrets getrennt von der übrigen Konfiguration. GitHub Actions übernimmt als SaaS-Dienst die CI/CD-Ausführung. Dadurch werden alle drei Cloud-Service-Modelle sichtbar genutzt, ohne zusätzliche Eigenbetriebs-Komplexität einzuführen.

2.3.1 Backend und Frontend

FastAPI wurde gegenüber Flask und Django gewählt, weil Pydantic-Modelle gleichzeitig Validierung und automatisch generierte OpenAPI-Spezifikation liefern und der Asynchronous Server Gateway Interface (ASGI)-Unterbau asynchrone Input/Output (I/O) für die mehrsekündigen Gemini-Aufrufe bereitstellt [3]. Im Frontend kommt React mit Vite zum Einsatz, weil Create React App (CRA) 2023 abgekündigt wurde und Vite als Nachfolger für Single Page Application (SPA)-Builds gilt [4]; Next.js hätte die bewusste Trennung zwischen statischem Frontend und containerisiertem Backend aufgeweicht.

2.3.2 Datenhaltung: Firestore vs. Cloud SQL und selbstbetriebene DB

Firestore wurde gegenüber Cloud SQL und einer selbstbetriebenen Datenbank bevorzugt. Nutzerprofile, Konversationen und Reisepläne sind dokumentenförmig; die Subcollection-Struktur `users/{uid}/conversations/{id}` entspricht direkt dem User Interface (UI)-Zugriffspfad. Das tägliche Rate-Limit erfordert einen atomaren Read-Modify-Write, den Firestore über Atomicity, Consistency, Isolation, Durability (ACID)-Transaktionen auf Dokumentenebene abdeckt [5]. Cloud SQL verursacht unabhängig von der Nutzung laufende Kosten [6], während Firestore mit seinem Free-Tier-Sockel den Projektbetrieb abdeckt [7].

2.3.3 Container-Orchestrierung: Docker Compose vs. Kubernetes (Google Kubernetes Engine (GKE))

Docker Compose wurde gegenüber Kubernetes gewählt, weil GKE eine Cluster-Management-Gebühr von rund \$73 pro Monat erhebt [8] und damit bereits das Budget der \$50 Education Credits überschreiten würde. Die im Projekt benötigten Kubernetes-Funktionen — horizontales Replizieren und Selbstheilung — übernimmt die Managed Instance Group mit Hypertext Transfer Protocol (HTTP)-Health-Check und Rolling-Replace [9].

2.3.4 Infrastructure as Code (IaC) und Konfigurationsmanagement: Terraform und Ansible

Terraform und Ansible waren durch die Modulvorgabe gesetzt; die Entscheidung betraf ihre Aufteilung. Terraform beschreibt deklarativ die Infrastruktur [10], Ansible arbeitet agentenlos

über Secure Shell (SSH) [11] und bleibt auf die persistente Monitoring-VM beschränkt; App-VMs werden stattdessen über Instance Template und Startup-Script bootstrapped.

2.3.5 Monitoring, Logging und Alerting

Prometheus und Grafana sind vorgegeben; Loki ergänzt den Stack, weil es sich nahtlos als Datenquelle in Grafana einbinden lässt. In der MIG-Topologie ist Prometheus besonders passend, weil die `gce_sd_config`-Service-Discovery neu erzeugte Instanzen automatisch als Scrape-Targets erkennt [12]. Node Exporter und Blackbox Exporter ergänzen Host-Metriken und aktive End-to-End-Prüfungen [13]. Eine OpenTelemetry-basierte Pipeline wird als Ausblick behandelt.

3 Architektur und Umsetzung

3.1 Gesamtarchitektur

Abbildung 1 zeigt die Architektur des Systems im Load-Balancer-basierten Betrieb mit $N > 1$ App-Instanzen. Der öffentliche Einstiegspunkt ist eine reservierte globale IP-Adresse, die von einem externen HTTP(S)-Load-Balancer gehalten wird. Dieser verteilt Traffic auf eine Managed Instance Group mit zwei App-VMs. Der Monitoring-Stack läuft getrennt auf einer persistenten VM mit eigener Persistent-Disk.

Die Darstellung ist nach Cloud-Service-Modellen gegliedert: SaaS umfasst die extern konsumierten Dienste GitHub Actions und Gemini, PaaS die verwalteten GCP-Dienste Firestore, Artifact Registry, Cloud Storage, Cloud Scheduler und Secret Manager, während IaaS die Compute- und Netzwerkkomponenten enthält. Dadurch werden drei Flussarten sichtbar: Nutzertraffic, Deployment/Bootstrap und Observability/Disaster Recovery. Gerade diese Trennung ist wichtig, weil ein Fehler im App-Pfad nicht gleichzeitig Monitoring, Backups oder die Wiederherstellung blockieren soll.

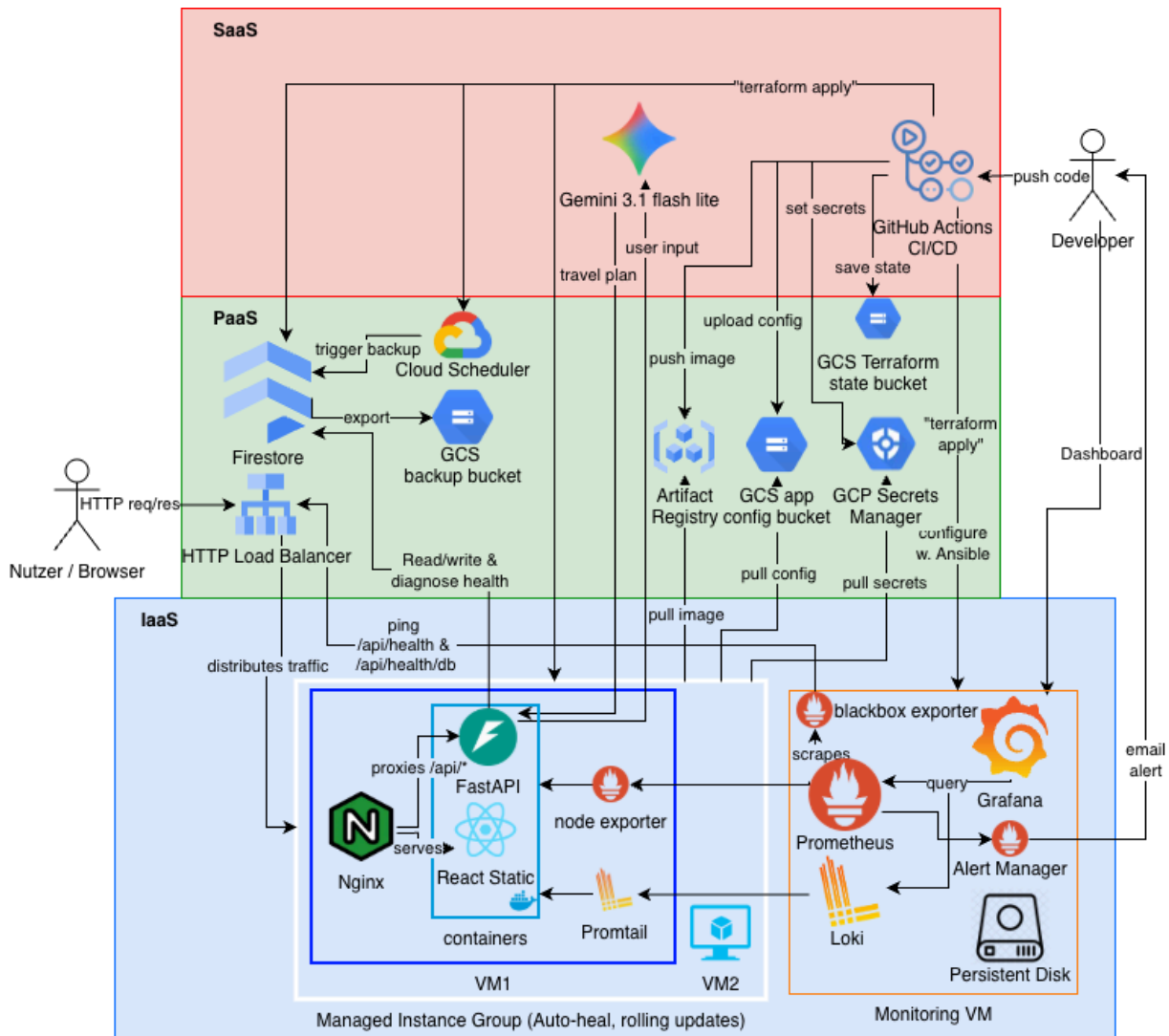


Abbildung 1: Gesamtarchitektur des Travel Assistant. Die Abbildung zeigt den produktiven Request-Pfad über Browser, Load Balancer und Managed Instance Group, den Deployment-Pfad über GitHub Actions, Artifact Registry und GCS-Konfiguration sowie die getrennten Monitoring- und Backup-Flüsse mit Prometheus, Loki, Grafana, Alertmanager, Cloud Scheduler und Firestore-Export.

3.2 Request-Pfad und Deployment-Flüsse

Abbildung 2 reduziert die Architektur auf drei operative Flüsse: Nutzertraffic, Rollout-Artefakte und Monitoring-Konfiguration. Produktions-Traffic läuft ausschließlich über den Load Balancer in die MIG; App-VMs sind keine öffentlichen Einstiegspunkte. Die Details des Rollout-Flusses — Artefakt-Bereitstellung, Bootstrap und Secret-Handhabung — behandelt Abschnitt 3.3.

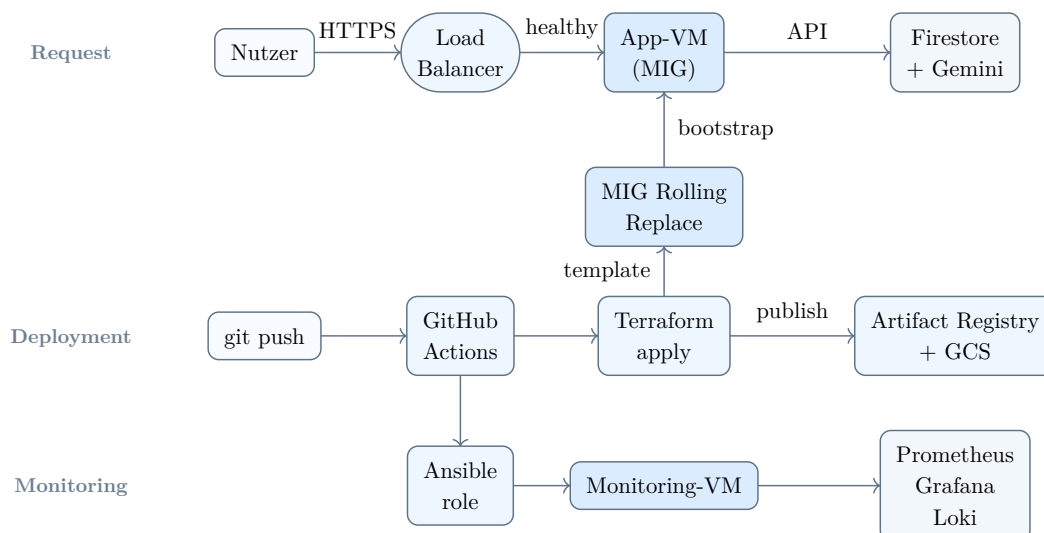


Abbildung 2: Getrennte Pfade für Laufzeit-Requests (oben) und Deployment (unten). Images und App-Konfiguration werden als Artefakte bereitgestellt; Ansible wird nur für die persistente Monitoring-VM verwendet.

3.3 Deployment-Modell

Terraform beschreibt die GCP-Infrastruktur (Load Balancer, MIG, Instance Template, Firestore, IAM, Buckets, Cloud Scheduler); App-VMs werden nicht per SSH konfiguriert, sondern starten über Instance Template und Startup-Script selbstständig. Das Startup-Script holt Docker-Compose, `.env` und Promtail-Konfiguration aus einem GCS-Bucket, zieht sensible Laufzeitwerte (Gemini-API-Key, JWT-Signing-Key) aus Google Secret Manager, authentifiziert Docker gegenüber Artifact Registry und startet den Compose-Stack mit Nginx, FastAPI, Promtail und Node Exporter. Container-Images werden mit dem Git-SHA getaggt, sodass pro Commit eine eindeutige Template-Version den MIG-Rollout auslöst.

Secrets und nicht-sensible Konfiguration sind bewusst getrennt: Der GCS-Bucket hält unkritische Werte (Projekt-ID, Loki-Endpoint, Image-Tag), Gemini-API-Key und JWT-Signing-Key liegen als benannte Secrets in Secret Manager. Der App-VM-Service-Account besitzt `roles/secretmanager.secretAccessor` pro Secret; Rotationen erfolgen per neuer Secret-Version plus MIG-Instance-Rotation, ohne Terraform-State oder Bucket-Inhalte zu berühren.

Ansible bleibt auf die langlebige Monitoring-VM beschränkt und verwaltet dort Prometheus, Grafana, Loki, Alertmanager und Nginx — Komponenten, die nicht mit jedem App-Rollout neu erzeugt werden.

3.4 Anwendungsumfang (Frontend und Backend)

Die Anwendung bildet einen durchgängigen Reiseplanungs-Workflow von der Konversation bis zur Trip-Verwaltung ab. Backend-seitig erzwingt `POST /api/ai/chat` ein striktes JSON-Envelope (`triptitle`, `response`, `recommendations`); bei Vertragsverletzungen folgt ein Reparatur-Retry, danach `HTTP 502` mit gekürztem Raw-Snippet zur Diagnose. `services/gemini.py` implementiert einen zweiphasigen Dialogfluss (Präferenzklärung, dann Itinerary-Generierung nach `GO`-Signal); Konversationen und Trips werden über dedizierte CRUD-Endpunkte persistiert.

Frontend-seitig rendert die Chat-Oberfläche Konversationsverwaltung (Suche, Pinning, Umbenennen, Löschen), Empfehlungschips und einen Retry-Flow; strukturierte Antworten werden als

Timeline-Karten mit Tagespunkten, Hotelvorschlägen und Budgetblock visualisiert. Reisepläne lassen sich direkt aus dem Chat speichern, später in denselben Trip zurückschreiben und als Markdown exportieren.

3.5 Monitoring-Stack

Der Monitoring-Stack läuft auf einer eigenen VM. Das ist bewusst gewählt, damit die Observability nicht mit der überwachten Infrastruktur verschwindet:

- **Prometheus** entdeckt Scrape-Targets über GCE-Service-Discovery. Ein statischer Target-Eintrag wäre bei einer rollierenden MIG fragil. Neben den FastAPI-Metriken werden auch Host-Metriken der App- und Monitoring-VMs über Node Exporter erfasst.
- **Blackbox Exporter** prüft den öffentlichen Load-Balancer-Pfad und darüber `GET /api/health/db` regelmäßig und exportiert `probe_success`. Der Endpoint wird also *aktiv* und *unabhängig* von App-Traffic beobachtet.
- **Grafana** lädt Dashboards automatisch aus einer provisionierten Verzeichnisstruktur.
- **Alertmanager** verarbeitet Prometheus-Alerts und kann bei gesetzten SMTP-Variablen E-Mail-Benachrichtigungen versenden.
- **Loki** nimmt Logs von Promtail entgegen, welches auf jeder App-VM Container- und System-Logs einsammelt.
- **Persistente Metriken und Logs** liegen auf einer separaten Persistent Disk, damit sie VM-Neustarts und Redeploys überleben. Für einen dauerhaften Betrieb wäre ein Terraform-`prevent_destroy` auf dieser Disk sinnvoll, weil es versehentliche Löschung von Monitoring-Historie bei `terraform destroy` verhindert. Dieser Schutz ist im gezeigten Setup nicht dauerhaft aktiviert, damit die Disk ohne zusätzlichen Terraform-Eingriff gelöscht werden kann (aus Ersparnisgründen).

Auf jeder App-VM laufen Promtail und Node Exporter als Sidecars neben Nginx und FastAPI: Promtail leitet Container- und Systemlogs an Loki weiter, Node Exporter stellt Host-Metriken für Prometheus bereit.

3.5.1 Alertierung und Betriebszugriff

Die Monitoring-VM stellt die Bedienoberflächen über einen lokalen Nginx bereit. Grafana ist unter `/grafana/` erreichbar; Prometheus und Alertmanager liegen unter `/prometheus/` beziehungsweise `/alertmanager/` und werden per Basic Auth geschützt, da beide Dienste selbst keine vollständige Zugriffsschicht für diesen Einsatzfall mitbringen. Dadurch bleibt Grafana als Dashboard leicht erreichbar, während die administrativeren Werkzeuge zusätzlich abgesichert sind.

Prometheus lädt seine Alert-Regeln aus einer provisionierten Datei und leitet ausgelöste Alarme an Alertmanager weiter. Die Regeln decken fünf Gruppen ab:

- **App-Verfügbarkeit:** `AppVMDown`, `NoHealthyAppVMs` und `AppPublicHealthCheckFailed` unterscheiden zwischen einzelner VM, fehlenden Scrape-Targets und einem tatsächlich fehlerhaften öffentlichen Load-Balancer-Pfad.
- **App-Performance:** Fehlerquote über 5% und p95-Latenz über zwei Sekunden werden als Warnungen erfasst.
- **Firestore-Integrität:** `FirestoreIntegrityCheckFailed` feuert nur, wenn der DB-Healthcheck fehlschlägt und die App-Instanz selbst erreichbar ist. Damit wird ein Datenproblem nicht mit einem reinen App-Ausfall verwechselt.

- **App-Ressourcen:** CPU-Auslastung und Root-Disk-Füllstand der App-VMs werden über Node Exporter überwacht.
- **Monitoring-Ressourcen:** Der Füllstand der persistenten Monitoring-Disk wird separat geprüft, weil ein voller Metrics-/Log-Speicher die Beobachtbarkeit selbst gefährden würde.

Alertmanager gruppiert Benachrichtigungen nach `alertname` und `severity`, wiederholt kritische Alarme häufiger und kann bei gesetzten SMTP-Variablen E-Mails versenden. Zusätzlich unterdrückt eine Inhibition den weniger aussagekräftigen `AppVMNodeExporterDown`, wenn gleichzeitig `AppVMDown` aktiv ist; bei einem kompletten VM-Ausfall reicht also der primäre App-Alarm.

4 Fokus-Feature: Disaster Recovery

4.1 Leitidee: zwei Recovery-Pfade

Das gewählte Fokus-Feature ist ein klar strukturiertes Disaster-Recovery-Konzept. Die zentrale Entscheidung: *Compute-Recovery* und *Datenrecovery* werden als zwei getrennte Recovery-Pfade behandelt, weil sie unterschiedliche Fehlerklassen adressieren (Abbildung 3).

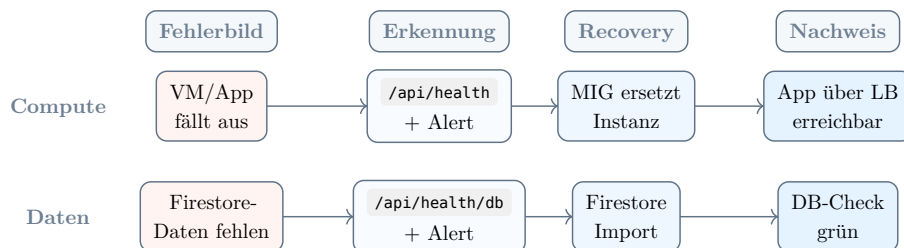


Abbildung 3: Zwei getrennte Recovery-Pfade mit jeweils eigener Erkennung, Wiederherstellung und Erfolgskontrolle.

Diese Trennung ergibt sich aus einer einfachen Beobachtung: Eine MIG heilt nur *Infrastruktur*, nicht *Daten*. Würde man ausschließlich auf MIG-Autohealing setzen, bliebe ein Szenario wie ein versehentlich gelöschttes Firestore-Dokument unbehandelt, obwohl die App-Instanzen technisch gesund sind.

4.2 Compute-Recovery über die MIG

Jede App-Instanz wird über einen HTTP-Health-Check gegen `/api/health` überwacht. Fällt eine Instanz aus, ersetzt die MIG sie automatisch anhand des hinterlegten Instance-Templates. Das Startup-Script bootstrapt die Ersatz-Instanz ohne manuellen Eingriff. Die globale Load-Balancer-IP bleibt dabei unverändert, weil sie nicht an eine konkrete VM gebunden ist.

4.3 Datenrecovery

4.3.1 Deterministischer Integritätscheck

Ein bloßer Ping auf Firestore würde einen Datenverlust nicht erkennen, da die Datenbank technisch weiterhin antwortet. Deshalb prüft ein dedizierter Endpoint `GET /api/health/db` konkret die Existenz *und* Pflichtfelder fester Sentinel-Dokumente:

- `users/sentinel-user` (Felder: `display_name`, `email`)
- `users/sentinel-user/trips/sentinel-trip` (Felder: `destination`, `start_date`, `end_date`, `status`)
- `analytics/system` (Felder: `seed_version`, `last_seeded_at`)

Fehlt eines dieser Dokumente oder ein Pflichtfeld, liefert der Endpoint `HTTP 500` mit einer strukturierten Fehlerliste und loggt ein Ereignis `db_integrity_error`. Ist Firestore selbst nicht erreichbar oder schlägt der Lesezugriff technisch fehl, wird derselbe HTTP-Status mit dem Grund `connectivity_error` zurückgegeben und als `db_connectivity_error` geloggt. Der Endpoint unterscheidet damit bewusst zwischen *Daten sind erreichbar, aber inhaltlich beschädigt* und *Datenbankzugriff ist technisch nicht möglich*.

Im Erfolgsfall antwortet der Endpoint mit `status: "healthy"` und `checked_documents: 3`. Im Fehlerfall enthält die Antwort `status: "unhealthy"`, einen maschinenlesbaren `reason` sowie die konkrete Liste der fehlenden Dokumente oder Felder. Dadurch kann der Recovery-Test nicht nur einen roten Zustand anzeigen, sondern auch begründen, welche Sentinel-Dokumente wiederhergestellt werden müssen.

Ein Blackbox Exporter fragt den Endpoint über den Prometheus-Job `db-health` ab und exportiert `probe_success`. Der Alert `FirestoreIntegrityCheckFailed` feuert nur, wenn `probe_success` fehlschlägt und die App-Instanz über den API-Scrape erreichbar bleibt — so wird ein Datenintegritätsfehler von einem App- oder VM-Ausfall getrennt. Alertmanager versendet bei konfigurierten SMTP-Variablen eine E-Mail und markiert den Alarm automatisch als resolved, sobald der Probe wieder `probe_success = 1` liefert; ein Grafana-Stat-Panel *Database Integrity* zeigt den Zustand zusätzlich kontinuierlich.

4.3.2 Backups und Restore

Die Backup-Strecke ist bewusst als täglicher Export umgesetzt: Der automatisierte Schutzpunkt entsteht einmal pro Tag, ergänzt durch manuelle Vorab-Backups für geplante Recovery-Tests. Diese Lösung ist einfach, prüfbar und passt zum Projektumfang, hat aber einen groben Recovery Point Objective (RPO): Änderungen zwischen zwei Scheduler-Läufen sind nicht durch den letzten geplanten Export abgedeckt. Firestore-Exporte eignen sich laut Google für das Wiederherstellen nach versehentlicher Löschung und werden über Cloud Storage abgelegt. Ein Export ist jedoch kein exakt zum Startzeitpunkt eingefrorener Snapshot [14].

Die Terraform-Konfiguration legt drei zusammenhängende Ressourcen an:

1. Ein GCS-Bucket `${project-id}-firestore-backups` mit einer 30-Tage-Lifecycle-Regel.
2. Ein dedizierter Service Account mit `datastore.importExportAdmin` auf Projektebene und `storage.objectCreator` auf dem Backup-Bucket (least privilege: nur Schreiben der Export-Artefakte, kein Löschen oder Lesen).
3. Ein Cloud-Scheduler-Job, der täglich um 03:00 Uhr Europe/Berlin die Firestore-Export-API mit OAuth-Token dieses Service Accounts aufruft und nach `gs://.../scheduled/` schreibt.

Zusätzlich existiert ein manuelles Backup-Skript `presentation/demo-scripts/firestore-backup.sh` für Vorab-Backups vor gezielten Recovery-Tests. Für den regulären manuellen Betrieb steht derselbe Vorgang zusätzlich als GitHub-Actions-Workflow `Manual Firestore Backup` bereit. Er ist über `workflow_dispatch` startbar, authentifiziert sich mit demselben GCP-Service-Account wie die Deployment-Pipeline und exportiert Firestore nach `gs://${project-id}-firestore-backups/manual/<timestamp>-run-<nr>-<label>`. Der erzeugte Backup-Pfad wird in der Job Summary ausgegeben, sodass der spätere Restore eindeutig auf einen konkreten Export zeigen kann. Für den Restore existiert analog der Workflow `Manual Firestore Restore`, der nach manueller Bestätigung entweder das jüngste manuelle

Backup oder einen explizit angegebenen `gs://`-Backup-Pfad importiert. Das lokale Script `presentation/demo-scripts/firestore-restore.sh` dient als Demo- und Fallback-Werkzeug.

4.3.3 Recovery-Zeit

Compute- und Datenpfad wurden jeweils über eine eigene Messreihe quantifiziert. Beide Messreihen sind als GitHub-Actions-Workflows (`perf-mig-recovery.yml`, `perf-dr-scale-test.yml`) automatisiert und reproduzierbar; sie schreiben die gemessenen Zeiten als JSON-Artefakte, die anschließend über `scripts/perf-plot.py` in die hier eingebundenen Diagramme überführt werden.

4.3.3.1 Compute-Recovery

Die Compute-Messreihe umfasst fünf aufeinanderfolgende Kill-and-Restore-Zyklen, in denen jeweils eine App-VM der MIG gestoppt und die Zeit bis zur vollständig wiederhergestellten Fleet gemessen wird. Die verbleibende zweite Instanz bedient währenddessen weiter Traffic, sodass die Messung gezielt das Autohealing-Verhalten abbildet und nicht einen Gesamtausfall.

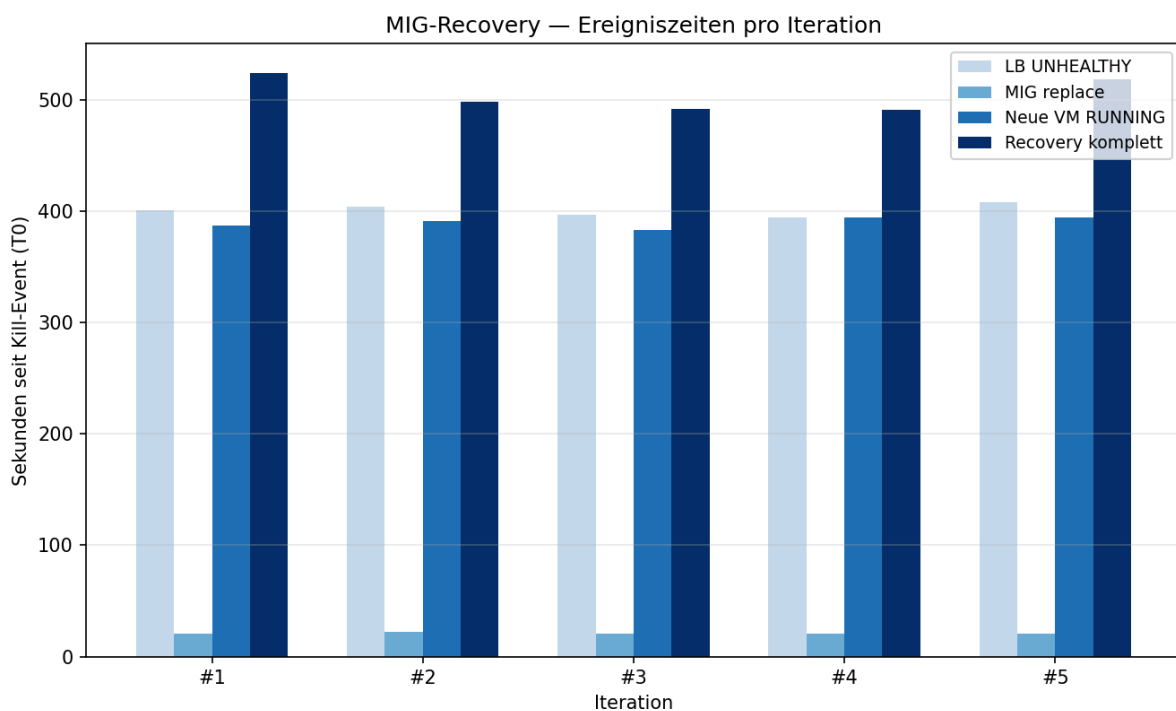


Abbildung 4: MIG-Recovery-Ereigniszeiten je Iteration über fünf Durchläufe.

Die Ergebnisse sind über alle Iterationen bemerkenswert stabil: der Median bis zur vollständigen Wiederherstellung liegt bei 498 s (≈ 8 min 18 s) bei einer Spannweite von nur 33 s (491–524 s). Der Ablauf gliedert sich in drei klar trennbare Phasen. Die MIG stößt die Ersatz-Provisionierung bereits nach median 21 s an, weil sie gestoppte Instanzen direkt über ihren VM-Status erkennt und nicht auf Health-Check-Failures wartet. Die neue VM erreicht nach median 391 s den Status RUNNING; dieser Schritt dominiert das Gesamtbudget und wird im Wesentlichen durch die Paketinstallation von Docker, die Artifact-Registry-Authentifizierung und den `docker compose pull` aus dem Startup-Script bestimmt. Nach weiteren rund 107 s ist die App antwortbereit und passt den Health-Check, sodass der Load Balancer Traffic auf die neue Instanz routet.

Die eingangs im Projekt angenommene rein Health-Check-basierte Erkennungszeit ($3 \times 30 \text{ s} = 90 \text{ s}$) beschreibt also nur den Sonderfall eines laufenden, aber abstürzenden Anwendungsprozesses. Im hier gemessenen Szenario eines kompletten VM-Ausfalls liegt die effektive Erkennung mehr als eine Größenordnung darunter und ist für die Recovery Time Objective (RTO)-Betrachtung vernachlässigbar; der dominierende Term ist der VM-Bootstrap. Ein vorgepacktes Image mit vorinstalliertem Docker und vorgepullten Container-Images würde diesen Term deutlich reduzieren und die Recovery überschlägig in den Bereich von fünf Minuten bringen.

4.3.3.2 Daten-Recovery

Für die Datenseite wurde eine zweite Messreihe mit drei Wiederholungen pro Größenklasse über 1 000, 10 000 und 100 000 synthetische Dokumente gefahren. Jede Iteration legt eine isolierte Collection an, exportiert sie nach Google Cloud Storage (GCS), löscht sie aus Firestore und importiert sie anschließend wieder, jeweils mit Zeitmessung pro Phase.

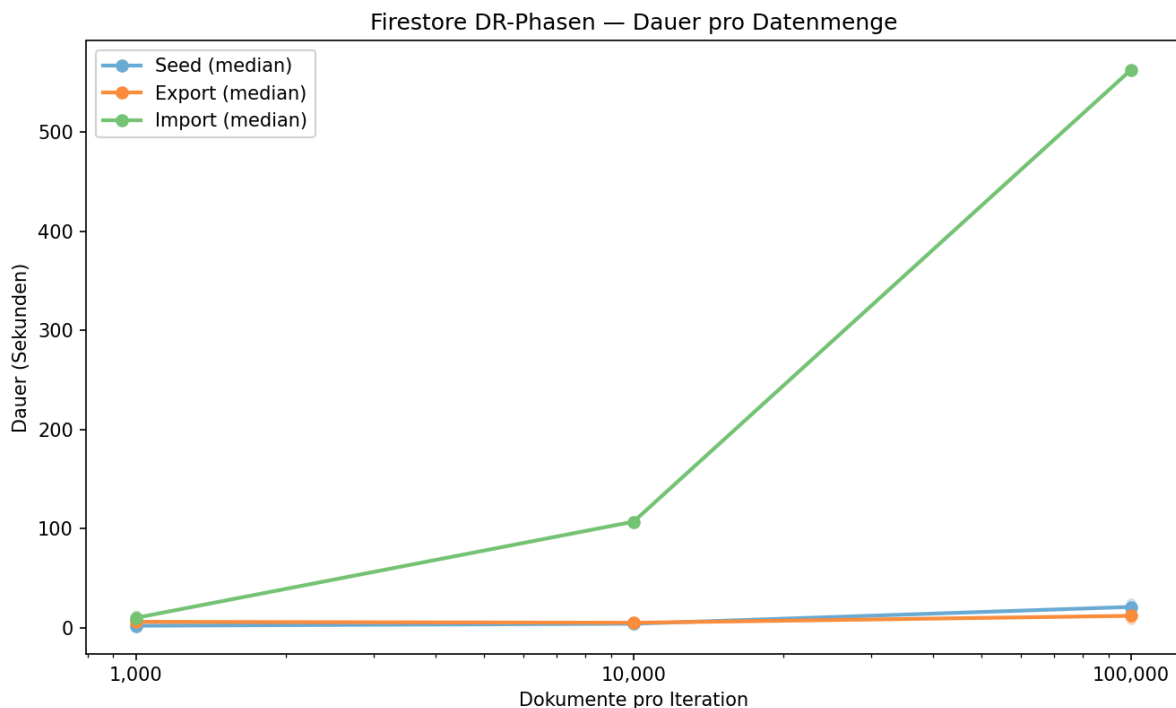


Abbildung 5: Firestore-Seed-, Export- und Import-Dauer in Abhängigkeit von der Datenmenge (x-Achse log, y-Achse linear in Sekunden). Transparente Punkte zeigen die einzelnen Iterationen, Linien die Mediane.

Die Messung zeigt ein klar asymmetrisches Bild. Der *Export* skaliert nur schwach und bleibt auch bei 100 000 Dokumenten unter 15 s, weil er serverseitig als Snapshot implementiert ist und im Wesentlichen Fixkosten aufweist. Der *Import* skaliert dagegen streng linear mit rund 5,6 ms pro Dokument: median 10 s bei 1 000, 107 s bei 10 000 und 563 s ($\approx 9 \text{ min } 23 \text{ s}$) bei 100 000 Dokumenten. Linear extrapoliert entspricht das bei einer Million Dokumenten einer Restore-Dauer von rund 94 Minuten.

Der Restore ist damit der dominierende Term im RTO-Budget der Datenrecovery, während der Export für die RTO-Planung praktisch irrelevant bleibt und nur die Backup-Häufigkeit (und damit den RPO) begrenzt. Solange der Datenbestand in der Größenordnung einiger zehntausend Dokumente bleibt, deckt der vollständige Import das Ziel einer Recovery innerhalb weniger Minuten ab. Bei stark wachsendem Datenvolumen würden die in Abbildung 3

skizzierten Teilstrategien — gezielter Restore betroffener Collection Groups oder ein Point-in-Time Recovery (PITR)-basiertes Zeitpunkt-Recovery — zum dominanten Hebel, weil ein vollständiger Import dann aus dem zulässigen RTO-Fenster fällt.

4.3.4 Kostenbewertung

Die Kosten lassen sich anhand der veröffentlichten Google-Cloud-Preise überschlägig berechnen. In Summe liegt der Grundbetrieb bei rund 38 bis 40 USD pro Monat, zuzüglich etwa 0,76 USD pro Monat je GiB produktiver Firestore-Daten für 30 tägliche Backup-Artefakte. Die genaue Herleitung ist im Anhang in Tabelle 3 dokumentiert. Tatsächliche Rechnungswerte können durch Rundung, Wechselkurse, Netzwerkverkehr, Artifact-Registry-Speicher und konkrete Exportgröße abweichen.

5 Ergebnisse und Diskussion

5.1 Rückblick auf den umgesetzten Systemumfang

Das umgesetzte System umfasst eine durchgängige, cloud-native End-to-End-Lösung aus React-Frontend, FastAPI-Backend, Firestore-Datenhaltung und Gemini-gestützter Assistenzlogik. Auf Infrastrukturebene wurden die zentralen Betriebsbausteine — MIG, Load Balancer, Artifact Registry, Secret Manager, Cloud Scheduler sowie ein separater Monitoring-Host mit Prometheus/Grafana/Loki — reproduzierbar über Terraform und Ansible bereitgestellt. Mit der CI/CD-Pipeline, automatisierten Tests und den nachgewiesenen Recovery-Pfaden (Compute-Autohealing, Firestore-Export/Import) ist damit nicht nur ein funktionaler Prototyp, sondern ein belastbarer Betriebs- und Wiederanlaufrahmen umgesetzt.

5.2 Testumfang und Methodik

Die automatisierten Tests laufen in GitHub Actions vor Build und Deployment und decken sowohl Backend- als auch Frontend-Verhalten ab. Im Backend prüfen `pytest` und der `FastAPI-TestClient` Authentifizierung, Chat- und Konversationsendpunkte, Trip-CRUD sowie `/api/health` und `/api/health/db`. Firestore und Gemini werden dabei gemockt, sodass die Suite deterministisch und ohne echte Cloud-Zugriffe läuft; für den DB-Integritätscheck existieren zusätzlich Negativtests für fehlende Referenzdaten und Firestore-Konnektivitätsfehler. Zusätzlich existieren **Gemini**-Integrationstests mit `pytest -m integration`, die reale Gemini-Aufrufe abdecken und als Teil der Backend-Testsuite laufen. Davon getrennt läuft nach dem Deployment ein eigener Public-App-Integrationsworkflow gegen die bereitgestellte Umgebung.

Im Frontend prüfen Vitest und React Testing Library komponentennahe Nutzerflüsse wie Login/Register-Formulare, Auth-Routing, Session-Management via AuthContext, Navigation, Chat-Fehlerfälle, das Speichern einer Antwort als Reiseplan sowie das Erstellen und Löschen von Trips. Die API-Schicht wird mit `axios-mock-adapter` gemockt; getestet werden damit UI-Verhalten, Zustandsübergänge und HTTP-Interceptor-Logik (Token-Injektion, 401-Handling), nicht visuelle Pixel-Regressionen. Ergänzend gatekeepen `ruff`, `mypy`, `pip-audit`, `tsc --noEmit`, ESLint, `vitest run --coverage` und `npm audit` formale Korrektheit, funktionale Korrektheit und Lieferkettensicherheit.

Die gemessene Zeilenabdeckung liegt bei rund 96 % im Backend und rund 77 % im Frontend; die detaillierten Coverage-Tabellen sind im Anhang in Tabelle 4 und Tabelle 5 dokumentiert. Zusätzlich läuft ein Integrationsworkflow gegen die öffentliche App-URL, der Health-Endpunkte,

Authentifizierung, Trip-CRUD und relevante Storage-Buckets mit echten GCP-Diensten prüft, aber bewusst auf Gemini-Aufrufe verzichtet. Vollständige End-to-End-Prüfungen von Load-Balancer-Verhalten, MIG-Autohealing oder realen Firestore-Imports bleiben aus Kosten- und Laufzeitgründen getrennten Demo- und Recovery-Nachweisen vorbehalten.

5.3 Grenzen der Lösung

- Das Restore ist manuell angestoßen; es gibt keine automatisierte RTO/RPO-getriebene Recovery-Entscheidung.
- Der belastbar nachgewiesene Daten-Recovery-Pfad basiert auf täglichen Firestore-Exporten; PITR ist zwar aktiviert, wurde im Projekt jedoch nicht als eigener, automatisierter Restore-Pfad getestet.
- Der Monitoring-Zugriff ist bewusst niedrigschwellig (öffentlich erreichbare Monitoring-VM, anonymer Grafana-Viewer), damit in der Präsentation ohne Zugangshürden zwischen App, Dashboard und Recovery-Nachweis gewechselt werden kann — für den Produktionsbetrieb wären IP-Restriktionen, Authentifizierungspflicht und HTTPS nötig.
- Logging läuft pragmatisch über Promtail nach Loki; verteilte Traces und eine vendor-neutrale Telemetrie-Pipeline fehlen.

5.4 Ausblick und Handlungsempfehlungen

5.4.1 Point-in-Time Recovery und partieller Restore

Firestore unterstützt nativ Point-in-Time Recovery (PITR) mit einem Recovery-Fenster von bis zu sieben Tagen und minutengenauen Lesezugriffen [15], und ist damit deutlich granularer als der hier umgesetzte tägliche Export. PITR ist in der Terraform-Konfiguration über `point_in_time_recovery_enablement = "POINT_IN_TIME_RECOVERY_ENABLED"` aktiviert; bei der hier produzierten Datenmenge liegen die Zusatzkosten im Cent-Bereich pro Monat. Ein konkreter zeitpunktbezogener Restore (Export mit `--snapshot-time` oder DB-Klon zu einem Zeitpunkt) ist aus Scope-Gründen im Rahmen dieser Arbeit nicht getestet oder als Workflow nachgewiesen worden — PITR steht damit als zusätzliche Recovery-Option zur Verfügung, der belastbare Nachweis beschränkt sich weiterhin auf die beiden gemessenen Pfade (MIG-Autohealing und vollständiger Firestore-Export/Import). Ein gezielter Restore einzelner Collection Groups wäre zusätzlich schneller als ein vollständiger Import und deckt den Fall kleiner, lokalisierter Datenfehler besser ab.

5.4.2 Robusterer Integritätscheck

Der Check ist bewusst deterministisch und erkennt vor allem das Fehlen fester Sentinel-Dokumente. Kleinere Datenverluste — gelöschte Trips, fehlende Pflichtfelder, Mengenabweichungen — würde er übersehen. Ein produktionsnäherer Check wäre mehrstufig: Konnektivität, versionierte Sentinel-Collection und fachliche Invarianten über Aggregationsabfragen (`count()`, `sum()`) [16] sowie Parity-Dokumente, die pro Collection Group erwartete Zähler oder Hashes in derselben Firestore-Transaktion mitpflegen [5]. Konzeptionell entspricht das etablierten Data-Quality-Dimensionen wie *Completeness*, *Consistency* und *Uniqueness*, wie sie etwa Dataplex Auto Data Quality formalisiert [17].

5.4.3 Parameter Manager für nicht-sensible Laufzeitkonfiguration

Die im GCS-Bucket verbliebene `.env` enthält nur deployment-spezifische Werte, die beim `terraform apply` ohnehin gerendert werden. Parameter, die sich ohne Rollout ändern sollen —

Feature-Flags, Gemini-Modell, Alert-Thresholds — ließen sich aus Google Parameter Manager beziehen, der Werte analog zu Secret Manager versioniert, aber ohne Secret-Garantien auskommt [18]. Im aktuellen Projektumfang existieren solche Werte nicht, weshalb diese Schicht überdimensioniert wäre.

6 Fazit

Das Projekt realisiert eine cloud-native Reiseplanungsanwendung mit klar getrennten Verantwortlichkeiten: Alle drei Cloud-Ebenen sind produktiv im Einsatz, die Infrastruktur wird vollständig über Terraform bereitgestellt, Ansible übernimmt die Konfiguration des persistenten Monitoring-Hosts, und das Monitoring-Konzept zeigt sowohl Infrastruktur- als auch Datengesundheit an.

Das Fokus-Feature *Disaster Recovery* unterscheidet klar zwischen Compute-Recovery und Datenrecovery und macht diese Trennung im Recovery-Ablauf sichtbar. Die eingeführte Unterscheidung zwischen Integritätscheck und reinem Connectivity-Ping sowie das zweistufige Backup-Konzept (geplant + manuell) machen den Recovery-Pfad nachvollziehbar. Der Load Balancer und die CI/CD-Pipeline ergänzen diese Architektur um einen stabilen öffentlichen Einstiegspunkt und reproduzierbare Rollouts.

Für zukünftige Iterationen bieten sich vor allem vier Richtungen an: *automatisiertes Restore* (RTO/RPO-getrieben), ein hybrider *Daily-Export-plus-PITR*-Ansatz für Firestore, eine *OpenTelemetry*-basierte Observability-Pipeline und ein *Multi-Region-Failover* für den Load Balancer. Die Trennung zwischen Terraform-gesteuertem App-Rollout und Ansible-gesteuertem Monitoring-Host bleibt dabei tragfähig und ermöglicht diese Erweiterungen, ohne die bestehende Deployment-Struktur zu brechen.

7 Anhang

7.1 Kostenrechnung

Die folgende Tabelle zeigt die überschlägige Kostenrechnung für den Grundbetrieb. Sie basiert auf den veröffentlichten On-Demand-Preisen und vereinfacht auf 730 Stunden pro Monat.

Posten	Annahme / Rechnung	Monatlich
Compute	$3 \times \text{e2-small} \times 730 \text{ h} \times 0,016752855 \text{ USD/h}$ [19]	36,69 USD
Persistent Disks	70 GiB <code>pd-standard</code> ; in <code>europa-west3</code> ohne anrechenbaren Compute-Free-Tier vollständig kostenpflichtig [20]	ca. 2,80 USD
Firestore	Kleine Projekt-Datenmenge innerhalb des Free Tier; Exporte verursachen dennoch Reads [7], [14]	ca. 0 USD
Backup Storage	$30 * D * 0,0253 \text{ USD}$ bei 30 täglichen Backups und Datenbankgröße <code>D</code> GiB [21]	ca. 0,76 USD/GiB
Cloud Scheduler	1 Job; innerhalb des Free Tier von drei Jobs, sonst 0,10 USD/Job [22]	ca. 0 USD
Summe	Grundbetrieb ohne variable Zusatzkosten	ca. 40–42 USD + 0,76 USD/GiB

Tabelle 3: Überschlägige monatliche Kostenrechnung.

7.2 Testabdeckung

Die folgenden Tabellen dokumentieren die gemessene Testabdeckung für Frontend und Backend. Die Frontend-Abdeckung stammt aus `npm run test:coverage` mit Vitest und V8-Coverage-Provider.

Datei	Statements	Branches	Functions	Lines	Nicht abgedeckte Zeilen
All files	74,83 %	59,66 %	66,8 %	76,98 %	
src	100 %	100 %	100 %	100 %	
App.tsx	100 %	100 %	100 %	100 %	
AuthContext.tsx	100 %	100 %	100 %	100 %	
src/components	69,96 %	59,92 %	61,19 %	72,26 %	
Chat.tsx	67,16 %	61,3 %	62,06 %	68,68 %	...-1078,1092-1093
Login.tsx	100 %	100 %	100 %	100 %	
Nav.tsx	100 %	100 %	100 %	100 %	
Register.tsx	100 %	100 %	100 %	100 %	
TripDetail.tsx	64,7 %	52,63 %	52,3 %	70,27 %	...397-413,427-496
TripPlanner.tsx	72,09 %	61,22 %	61,11 %	72,36 %	...246,267,296-299
src/services	100 %	88,88 %	100 %	100 %	
api.ts	100 %	88,88 %	100 %	100 %	15
src/utis	92,1 %	50 %	100 %	94,36 %	
tripMarkdown.ts	92,1 %	50 %	100 %	94,36 %	13-21, 41

Tabelle 4: Frontend-Testabdeckung mit Vitest und V8-Coverage.

Die Backend-Abdeckung stammt aus der `pytest`-Testsuite mit `pytest-cov`.

Datei	Statements	Miss	Coverage	Fehlend
main.py	34	0	100 %	
metrics.py	8	0	100 %	
routers/__init__.py	0	0	100 %	
routers/ai.py	178	21	88 %	48-49, 52, 63, 65, 67, 69, 71, 85, 92, 102-103, 122, 147-153, 203
routers/auth.py	62	0	100 %	
routers/health.py	37	3	92 %	21, 62, 84
routers/trips.py	66	0	100 %	
services/__init__.py	0	0	100 %	
services/firestore_client.py	7	0	100 %	
services/gemini.py	69	5	93 %	180-191
services/jwt_auth.py	24	0	100 %	
tests/__init__.py	0	0	100 %	
tests/conftest.py	22	0	100 %	
tests/test_ai.py	154	0	100 %	
tests/test_auth.py	55	0	100 %	
tests/integration/test_gemini.py	77	5	94 %	96-98, 155, 181
tests/test_health.py	62	3	95 %	86, 93, 98
tests/test_services.py	111	0	100 %	
tests/test_trips.py	86	0	100 %	
TOTAL	1052	37	96 %	

Tabelle 5: Backend-Testabdeckung mit pytest-cov.

Referenzen

- [1] Google, „Gemini 3.1 Flash-Lite: Built for intelligence at scale“. Zugegriffen: 20. April 2026. [Online]. Verfügbar unter: <https://blog.google/innovation-and-ai/models-and-research/gemini-models/gemini-3-1-flash-lite/>
- [2] Google, „Google AI Studio — API Rate Limits (Free Tier)“. Zugegriffen: 20. April 2026. [Online]. Verfügbar unter: <https://aistudio.google.com/app/rate-limit>
- [3] Sebastian Ramirez, „FastAPI — Features: On par with NodeJS and Go, automatic OpenAPI, Pydantic-based validation“. Zugegriffen: 20. April 2026. [Online]. Verfügbar unter: <https://fastapi.tiangolo.com/features/>
- [4] Vite, „Why Vite — Problems and how Vite addresses them“. Zugegriffen: 20. April 2026. [Online]. Verfügbar unter: <https://vite.dev/guide/why.html>
- [5] Google Cloud, „Firestore: Transactions and Batched Writes“. Zugegriffen: 20. April 2026. [Online]. Verfügbar unter: <https://cloud.google.com/firestore/docs/manage-data/transactions>
- [6] Google Cloud, „Cloud SQL Pricing“. Zugegriffen: 20. April 2026. [Online]. Verfügbar unter: <https://cloud.google.com/sql/pricing>
- [7] Google Cloud, „Firestore Pricing (Free Tier and Spark Plan)“. Zugegriffen: 20. April 2026. [Online]. Verfügbar unter: <https://cloud.google.com/firestore/pricing>
- [8] Google Cloud, „GKE Pricing — Cluster Management Fee“. Zugegriffen: 20. April 2026. [Online]. Verfügbar unter: <https://cloud.google.com/kubernetes-engine/pricing>
- [9] Google Cloud, „Compute Engine: Managed Instance Groups“. Zugegriffen: 20. April 2026. [Online]. Verfügbar unter: <https://cloud.google.com/compute/docs/instance-groups>

- [10] HashiCorp, „Terraform Google Provider Documentation“. Zugegriffen: 20. April 2026. [Online]. Verfügbar unter: <https://registry.terraform.io/providers/hashicorp/google/latest/docs>
- [11] Red Hat, „How Ansible Works — Agentless architecture over SSH“. Zugegriffen: 20. April 2026. [Online]. Verfügbar unter: <https://www.ansible.com/how-ansible-works/>
- [12] Prometheus Authors, „Prometheus Configuration: gce_sd_config“. Zugegriffen: 20. April 2026. [Online]. Verfügbar unter: https://prometheus.io/docs/prometheus/latest/configuration/configuration/#gce_sd_config
- [13] Prometheus Authors, „Blackbox Exporter“. Zugegriffen: 20. April 2026. [Online]. Verfügbar unter: https://github.com/prometheus/blackbox_exporter
- [14] Google Cloud, „Firestore: Export and Import Data“. Zugegriffen: 20. April 2026. [Online]. Verfügbar unter: <https://cloud.google.com/firestore/docs/manage-data/export-import>
- [15] Google Cloud, „Firestore: Point-in-Time Recovery“. Zugegriffen: 20. April 2026. [Online]. Verfügbar unter: <https://cloud.google.com/firestore/docs/use-pitr>
- [16] Google Cloud, „Firestore: Summarize Data with Aggregation Queries“. Zugegriffen: 22. April 2026. [Online]. Verfügbar unter: <https://cloud.google.com/firestore/native/docs/query-data/aggregation-queries>
- [17] Google Cloud, „Dataplex Universal Catalog: Auto Data Quality Overview“. Zugegriffen: 22. April 2026. [Online]. Verfügbar unter: <https://cloud.google.com/dataplex/docs/auto-data-quality-overview>
- [18] Google Cloud, „Parameter Manager — Overview“. Zugegriffen: 24. April 2026. [Online]. Verfügbar unter: <https://cloud.google.com/secret-manager/parameter-manager/docs/overview>
- [19] Google Cloud, „Compute Engine VM Instance Pricing“. Zugegriffen: 22. April 2026. [Online]. Verfügbar unter: <https://cloud.google.com/compute/vm-instance-pricing>
- [20] Google Cloud, „Compute Engine Disk and Image Pricing“. Zugegriffen: 22. April 2026. [Online]. Verfügbar unter: <https://cloud.google.com/compute/disks-image-pricing>
- [21] Google Cloud, „Cloud Storage Pricing“. Zugegriffen: 22. April 2026. [Online]. Verfügbar unter: <https://cloud.google.com/storage/pricing>
- [22] Google Cloud, „Cloud Scheduler Pricing“. Zugegriffen: 22. April 2026. [Online]. Verfügbar unter: <https://cloud.google.com/scheduler/pricing>
- [23] Google Cloud, „External HTTP(S) Load Balancing Overview“. Zugegriffen: 20. April 2026. [Online]. Verfügbar unter: <https://cloud.google.com/load-balancing/docs/https>
- [24] Google Cloud, „Dataplex Universal Catalog: DataQualityRule“. Zugegriffen: 22. April 2026. [Online]. Verfügbar unter: <https://cloud.google.com/dataplex/docs/reference/rest/v1/DataQualityRule>
- [25] Google Cloud, „Cloud Scheduler Documentation“. Zugegriffen: 20. April 2026. [Online]. Verfügbar unter: <https://cloud.google.com/scheduler/docs>
- [26] Red Hat and Ansible Community, „Ansible Documentation“. Zugegriffen: 20. April 2026. [Online]. Verfügbar unter: <https://docs.ansible.com/>

- [27] Prometheus Authors, „Prometheus Documentation“. Zugegriffen: 20. April 2026. [Online]. Verfügbar unter: <https://prometheus.io/docs/introduction/overview/>
- [28] Grafana Labs, „Grafana and Loki Documentation“. Zugegriffen: 20. April 2026. [Online]. Verfügbar unter: <https://grafana.com/docs/>
- [29] Sebastian Ramirez, „FastAPI Documentation“. Zugegriffen: 20. April 2026. [Online]. Verfügbar unter: <https://fastapi.tiangolo.com/>
- [30] TechEmpower, „Web Framework Benchmarks — Round 22“. Zugegriffen: 20. April 2026. [Online]. Verfügbar unter: <https://www.techempower.com/benchmarks/>
- [31] Grafana Labs, „Loki: Like Prometheus, but for logs“. Zugegriffen: 20. April 2026. [Online]. Verfügbar unter: <https://grafana.com/oss/loki/>
- [32] HashiCorp, „Terraform vs. Alternatives — Cloud-agnostic infrastructure as code“. Zugegriffen: 20. April 2026. [Online]. Verfügbar unter: <https://developer.hashicorp.com/terraform/intro/vs>